

说明

FMC（Flexible Memory Controller，可变存储控制器）是微控制器中用于扩展外部存储器的外设模块。FMC 支持驱动包括 SRAM、SDRAM、NOR FLASH 及 NAND FLASH 在内的多种存储器类型，其中 SDRAM 控制器是 FMC 的核心功能之一，为需要大容量运行内存的应用场景提供了硬件基础，本应用手册主要介绍了如何使用 FMC 驱动 SDRAM，包括硬件设计和软件配置以及如何同时挂在两个 SDRAM 使用。本设计仅支持普通 SDRAM，不支持 DDR 类型。

适用范围

适用范围	系列
通用 MCU	CH32H417

目录

说明 1

目录 2

表格索引..... 2

第 1 章 基本工作原理 1

 1. 1 信号线说明 1

 1. 2 SDRAM 存储结构..... 1

 1. 3 SDRAM 关键时序参数..... 1

第 2 章 硬件设计指南 2

 2. 1 FMC 与 SDRAM 引脚连接对照 2

 2. 2 硬件设计要点 2

第 3 章 软件配置指南 3

 3. 1 配置流程概述 3

第 4 章 常见问题与调试技巧 6

 4. 1 初始化失败排查..... 6

 4. 2 读写失败排查 6

 4. 3 读写速度实测 6

历史版本..... 7

声明..... 8

表格索引

信号线说明..... 1

引脚对应连接 2

第1章 基本工作原理

1.1 信号线说明

SDRAM 芯片的外部信号线可分为以下几类，掌握各信号线的功能是正确进行硬件设计和软件配置的基础

表 1-1 信号线说明

信号线	类型	说明
CLK	O	同步时钟信号，所有输入信号在 CLK 上升沿被采集
CKE	O	时钟使能信号，禁止时钟时 SDRAM 启动自刷新操作
CS#	O	片选信号，低电平有效
CAS#	O	列地址选通，为低电平时地址线表示列地址
RAS#	O	行地址选通，为低电平时地址线表示行地址
WE#	O	写入使能，低电平有效
DQM[0:1]	O	数据输入/输出掩码信号
BA[0:1]	O	Bank 地址输入，选择要控制的内部 Bank
A[0:12]	O	地址输入（行列复用）
DQ[0:15]	I/O	数据输入/输出信号

1.2 SDRAM 存储结构

SDRAM 内部包含多个存储阵列（Bank），每个 Bank 由行（Row）和列（Column）寻址。以常见的 W9825G6KH 为例，其内部包含 4 个 Bank，每个 Bank 具有 13 位行地址和 9 位列地址，数据宽度为 16 位，总容量为 $4 \times 2^{13} \times 2^9 \times 16\text{bit} = 32\text{MB}$ 。

1.3 SDRAM 关键时序参数

SDRAM 的时序参数是驱动配置的核心，以下参数必须与 SDRAM 数据手册严格对应：

tRCD（RAS-to-CAS Delay）：行有效命令与读/写列命令之间的延迟周期数，一般为 2~3 个时钟周期。

tRP（Row Precharge Delay）：预充电命令到下一个有效命令之间的延迟周期数。

tWR（Write Recovery Time）：写操作结束到预充电命令之间的延迟周期数。

tRC（Row Cycle Delay）：连续两个行有效命令之间的最小间隔，以及刷新命令与激活命令之间的间隔。

tRAS（Self Refresh Time）：自刷新操作的最小 SDRAM 时钟间隔。

CAS Latency：发送列地址（READ 命令）之后到数据开始输出的 SDRAM 时钟周期数，通常为 2 或 3，SDRAM 控制器配置必须与 SDRAM 模式寄存器的设置保持一致。

刷新周期：SDRAM 需要周期性刷新以维持数据，标准要求每 64ms 刷新全部行。刷新周期计算公式为：刷新间隔 = $64\text{ms} \div (\text{行数})$

第2章 硬件设计指南

2.1 FMC 与 SDRAM 引脚连接对照

CH32 的 FMC 外设通过专用引脚与 SDRAM 芯片连接，下表列出了关键的引脚对应关系：

表 2-1 引脚对应连接

FMC 引脚	SDRAM 引脚	说明
FMC_A[12:0]	A[12:0]	行/列地址线
FMC_A[15:14]	BA[1:0]	Bank 地址线
FMC_D[15:0]	DQ[15:0]	数据线（16 位宽度）
FMC_NBL[1:0]	DQM[1:0]	数据掩码信号
FMC_SDCLK	CLK	同步时钟信号
FMC_SDNWE	WE#	写入使能
FMC_NRAS	RAS#	行地址选通
FMC_NCAS	CAS#	列地址选通
FMC_SDNE[1:0]	CS#	芯片使能（片选）
FMC_SDCKE[1:0]	CKE	时钟使能

注释：FMC_SDNE 和 FMC_SDCKE 各有两条信号线，分别对应两个独立的存储区域。使用编号为 0 的信号线组对应存储器区域 1（首地址 0xC000 0000），使用编号为 1 的信号线组对应存储器区域 2（首地址 0xD000 0000）

2.2 硬件设计要点

1. 数据总线宽度匹配：如果 SDRAM 芯片为 16 位宽度，只需连接 FMC_D[15:0] 前 16 条数据线，同时在 FMC 配置寄存器中将数据宽度配置为 16 位。
2. 时钟信号：FMC_SDCLK 的时钟频率可通过 FMC_SDCR1 寄存器的 SDCLK 位配置为 HCLK 的 1/2 或 1/3，与 SDRAM 通讯的同步时钟最高频率为 100MHz。
3. NBL（字节掩码）信号：对于 16 位数据总线，需要 NBL0 和 NBL1 两个掩码信号；8 位总线仅需 NBL0；32 位总线需要使用全部 4 个 NBL。
4. PCB 布局建议：
 - i. SDRAM 应尽量靠近 CH32 放置，缩短走线长度
 - ii. 时钟线 FMC_SDCLK 走线长度应尽量与数据/地址线匹配
 - iii. 注意信号完整性和电源去耦处理
 - iv. 避免 SDRAM 与 NAND 等设备同时占用 FMC 总线导致总线冲突

第3章 软件配置指南

3.1 配置流程概述

FMC 驱动 SDRAM 的软件配置按以下步骤进行：

- 1) 引脚时钟配置：配置 FMC GPIO 口和时钟源使能

```
RCC_HBPeriphClockCmd(RCC_HBPeriph_FMC,ENABLE);
```

```
GPIO_Config();
```

- 2) FMC 控制器配置：配置 SDRAM 控制寄存器和时序寄存器，如果需要同时使用两个 SDRAM，则需要分开两次配置 bank5 和 bank6,且 bank6 的配置参数与 bank5 均为一致，无法单独配置。

```
FMC_SDRAM_InitTypeDef SDRAMInitStructure={0};
```

```
FMC_SDRAM_TimingTypeDef SDRAM_Timing={0};
```

```
SDRAMInitStructure.FMC_Bank=FMC_Bank5_SDRAM;           //选择 bank
```

```
SDRAMInitStructure.FMC_ColumnBitsNumber=FMC_ColumnBitsNumber_9;    //SDRAM 的行数
```

```
SDRAMInitStructure.FMC_RowBitsNumber=FMC_ROWBitsNumber_12;         //SDRAM 的列数
```

```
SDRAMInitStructure.FMC_MemoryDataWidth=FMC_MemoryDataWidth_32;     //SDRAM 的位宽
```

```
SDRAMInitStructure.FMC_InternalBankNumber=FMC_InternalBankNumber_4; //SDRAM 的 bank 数
```

```
SDRAMInitStructure.FMC_CASLatency=FMC_CASLatency_2CLk;             //CAS 延迟
```

```
SDRAMInitStructure.FMC_WriteProtection=FMC_WriteProtection_Disable;
```

```
SDRAMInitStructure.FMC_SDClockPeriod=1; //时钟分频 SDRAM 的 CLK=HCLK/ SDClockPeriod
```

```
SDRAMInitStructure.FMC_ReadBurst=FMC_ReadBurst_Enable;             //是否使用突发读模式
```

```
SDRAMInitStructure.FMC_ReadPipeDelay=FMC_ReadPipeDelay_none;
```

```
SDRAMInitStructure.FMC_PHASE_SEL=0xF;
```

```
SDRAM_Timing.FMC_LoadToActiveDelay=2;
```

```
SDRAM_Timing.FMC_ExitSelfRefreshDelay=8;
```

```
SDRAM_Timing.FMC_SelfRefreshTime=6;
```

```
SDRAM_Timing.FMC_RowCycleDelay=6;
```

```
SDRAM_Timing.FMC_WriteRecoveryTime=2;
```

```
SDRAM_Timing.FMC_RPDelay=2;
```

```
SDRAM_Timing.FMC_RCDDelay=2;
```

```
SDRAMInitStructure.FMC_SDRAM_Timing = &SDRAM_Timing;
```

```
FMC_SDRAM_Init(&SDRAMInitStructure);
```

- 3) SDRAM 初始化序列：发送命令完成 SDRAM 芯片上电初始化，如果需要同时使用两块 SDRAM，则需要对两个 BNAK 都进行上电初始化，如果使用两个 SDRAM 存储区域，则必须通过将 CTB1 和 CTB2 位置 1 来向两个器件同时发送自刷新和 PALL 命令，否则该命令将被忽略。如果只使用一个 SDRAM 存储区域，并通过将其关联的 CTB 位置 1 来发出命令，则未使用存储区域的另一个 CTB 位必须保持为 0。

```
FMC_Bank5_6->MISC|=(1<<16);.
```

```
SDRAM_Send_Cmd(0,FMC_SDRAM_CMD_CLK_ENABLE,1,0);
```

```
Delay_Us(500);
```

```
SDRAM_Send_Cmd(0,FMC_SDRAM_CMD_PALL,1,0);
```

```
SDRAM_Send_Cmd(0,FMC_SDRAM_CMD_AUTOREFRESH_MODE,8,0);
```

```

temp=(u32)SDRAM_MODEREG_BURST_LENGTH_1 |
SDRAM_MODEREG_BURST_TYPE_SEQUENTIAL |
SDRAM_MODEREG_CAS_LATENCY_2 |
SDRAM_MODEREG_OPERATING_MODE_STANDARD |
SDRAM_MODEREG_WRITEBURST_MODE_SINGLE;
SDRAM_Send_Cmd(0,FMC_SDRAM_CMD_LOAD_MODE,1,temp);
FMC_SDRAM_SetRefreshCnt(700);
}

```

4) 读写测试验证

读写可以直接通过地址映射去 U8/U16/U32 的访问

```

for(i=0; i<5120; i++){
    *(u32*)(Bank5_SDRAM_ADDR + 4*i) = i;
}

printf("Read data:\n");
for(i=0; i<5120; i++){
    p=*(u32*)(Bank5_SDRAM_ADDR + 4*i);
    printf("%08x ", p);
}

```

读写也可以直接通过 DMA 的方式进行以达到速度优化的目的，选择最大位宽 256，开启读增强/预取模式：Enhance_read_mode 置 1，注意突发读和增强读不可以同时开启，否则将无效

```

/*****
* @fn      FMC_SDRAM_WriteBuffer
*
* @brief   Write buffer.
*
* @return  none
*/

void FMC_SDRAM_WriteBuffer_DMA(u32 *pBuffer,u32 WriteAddr,u32 n)
{
    DMA_InitTypeDef DMA_InitStructure = {0};
    RCC_HBPeriphClockCmd(RCC_HBPeriph_DMA1, ENABLE);
    DMA_Cmd(DMA1_Channel3, DISABLE);
    DMA_StructInit(&DMA_InitStructure);
    DMA_InitStructure.DMA_PeripheralBaseAddr= (volatile u32)(Bank5_SDRAM_ADDR+WriteAddr);
    DMA_InitStructure.DMA_Memory0BaseAddr = (volatile u32)pBuffer;
    DMA_InitStructure.DMA_DIR = DMA_DIR_PeripheralDST;
    DMA_InitStructure.DMA_BufferSize = n/8;
    DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Enable;
    DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Enable;
    DMA_InitStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_256;
    DMA_InitStructure.DMA_MemoryDataSize = DMA_MemoryDataSize_256;
    DMA_InitStructure.DMA_Mode = DMA_Mode_Normal;
    DMA_InitStructure.DMA_Priority = DMA_Priority_VeryHigh;
    DMA_InitStructure.DMA_M2M = DMA_M2M_Enable;
    DMA_Init(DMA1_Channel3, &DMA_InitStructure);
}

```

```

DMA_ClearFlag(DMA1,DMA1_FLAG_TC3);
DMA_Cmd(DMA1_Channel3, ENABLE);
while(DMA_GetFlagStatus(DMA1,DMA1_FLAG_TC3) == RESET)
{
}
}

/*****
* @fn      FMC_SDRAM_ReadBuffer
*
* @brief   Read buffer.
*
* @return  none
*/

void FMC_SDRAM_ReadBuffer_DMA(u32 *pBuffer,u32 ReadAddr,u32 n)
{
    DMA_InitTypeDef DMA_InitStructure = {0};
    DMA_Cmd(DMA1_Channel3, DISABLE);
    RCC_HBPeriphClockCmd(RCC_HBPeriph_DMA1, ENABLE);
    DMA_StructInit(&DMA_InitStructure);
    DMA_InitStructure.DMA_PeripheralBaseAddr = (volatile u32)(Bank5_SDRAM_ADDR+ReadAddr);
    DMA_InitStructure.DMA_Memory0BaseAddr = (volatile u32)pBuffer;
    DMA_InitStructure.DMA_DIR = DMA_DIR_PeripheralSRC;
    DMA_InitStructure.DMA_BufferSize = n/8;
    DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Enable;
    DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Enable;
    DMA_InitStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_256;
    DMA_InitStructure.DMA_MemoryDataSize = DMA_MemoryDataSize_256;
    DMA_InitStructure.DMA_Mode = DMA_Mode_Normal;
    DMA_InitStructure.DMA_Priority = DMA_Priority_VeryHigh;
    DMA_InitStructure.DMA_M2M = DMA_M2M_Enable;
    DMA_Init(DMA1_Channel3, &DMA_InitStructure);
    DMA_ClearFlag(DMA1,DMA1_FLAG_TC3);
    DMA_Cmd(DMA1_Channel3, ENABLE);
    while(DMA_GetFlagStatus(DMA1,DMA1_FLAG_TC3) == RESET)
    {
    }
}

FMC_SDRAM_WriteBuffer_DMA(TxBuffer,WRITE_READ_ADDR,BUFFER_SIZE);
FMC_SDRAM_ReadBuffer_DMA(RxBuffer, WRITE_READ_ADDR, BUFFER_SIZE);

```

第4章 常见问题与调试技巧

4.1 初始化失败排查

当 SDRAM 初始化失败时，应从以下方面逐层排查：

1. 硬件连接检查：确认 FMC 接口引脚是否正确配置，检查 SDRAM 电源电压（通常 3.3V）是否稳定
2. 时钟配置验证：确保 FMC 时钟源来自 HCLK，分频系数计算正确
3. 时序参数核对：对照 SDRAM 数据手册检查 TRCD、TWR、TRP、TRC 等时序参数是否与手册一致
4. 使用示波器检测：观测 FMC_SDCLK 信号是否稳定，检查控制信号是否按预期输出

4.2 读写失败排查

1. 若连续读写 SDRAM 时出错，但是增加延时后正常，通常是时序配置过快导致的。可尝试降低 FMC 时钟频率，再逐步调整时序参数相位等
2. 确保正确配置了 SDRAM 的 Bank 选择、行列地址位数和数据总线宽度
3. 确认 MCU 端使用的片选信号与 FMC Bank 配置对应
4. 检查 SDRAM 的地址空间是否与数据手册中的容量匹配

4.3 读写速度实测

通过大批量的读写来测定速度，配置独立的 IO 口，读/写操作开始拉低 IO，读/写操作结束拉高 IO，通过计算得出 SDRAM 的实际读写速度：

32 位 SDRAM，在 SDRAM_CLK=100MHz 时，DMA 方式 256 位宽下实际读写速度为 380MB/S；与理论上的 400MB/S 仅存在部分损耗。

历史版本

日期	版本	变更内容
2026/3/17	V1.0	初版发行

声明

本手册仅供参考。作者在不另行通知的情况下，可随时进行修改。